Tidy Tag Cleaning Platform: Objective, Method and Conclusion

Project Objective

Tidy Tag is a booking platform for a house and lease cleaning business: customers get a live price and submit a booking, and the business is notified the moment a request arrives.

The project set out to meet five goals:

- **Live, config-driven pricing.** Property types, cleaning tiers, add-ons, frequency discounts and room limits live in a JSON file, so the owner changes prices without editing code.
- **Trustworthy quotes.** The server recalculates every price, so a customer cannot alter a quote in the browser.
- **Social proof.** Show the business's Google reviews on the site.
- **A real booking pipeline.** Validate each request, store it, and email both the business and the customer.
- **Simple, cheap operation.** One Node/Express app on Google Cloud Run that still works locally with no credentials.

Method

The platform is one Node 20 / Express app that serves a no-build vanilla HTML/CSS/JS frontend and four JSON API routes. This section is based on the README and packaging files; the server and frontend source were not reviewed.

Stack

Layer	Choice	Role
Frontend	Vanilla HTML, CSS, JS (`public/`)	Fetches `/api/config` and builds every pill, tier card and add-on from it
Server	Express 4, cors, dotenv	Static files plus the API routes
Email	Nodemailer 6 over SMTP	Booking and confirmation emails; logged to console when unconfigured
Storage	Firestore (`@google-cloud/firestore` 7)	Saves bookings; no-op when not configured
Reviews	Google Places API	Cached reviews, with bundled sample reviews as fallback
Hosting	Docker (`node:20-slim`) on Cloud Run	Port 8080, Secret Manager for secrets

API routes

Route	Purpose
`GET /api/config`	Returns business and pricing config to the frontend
`POST /api/quote`	Server-side authoritative price calculation
`GET /api/reviews`	Google reviews (cached) or fallback reviews
`POST /api/bookings`	Validates, prices, saves, then emails the team and the customer

Approach

1. **Configuration over code.** Business details and all pricing rules sit in `business.json` and `pricing.json`; a restart applies changes.
2. **One pricing source of truth.** A shared `pricing.js` calculator is used by both the quote and booking routes, and the booking route reprices every request itself.
3. **Graceful degradation.** With no `.env` values the site, quotes and bookings still work; emails are logged and storage is skipped, so development needs no live credentials.
4. **Deployment.** Build the image with Cloud Build, deploy to Cloud Run, keep `SMTP_PASS` and the Places key in Secret Manager, and use Application Default Credentials for Firestore.
5. **Review.** The attached files were checked for runtime, build, security and operations issues; the findings are in the Conclusion.

Conclusion

The design meets its objective: pricing is config-driven and server-verified, bookings reach the business by email, and the app runs without credentials in development. It is not yet ready for public launch, because four issues should be fixed first.

Strengths

- Server-authoritative pricing, so quotes cannot be tampered with.
- Secrets are kept out of the env file and routed through Secret Manager.
- Graceful fallbacks for email, storage and reviews.